

DEEP LEARNING

Unsupervised Learning Part II

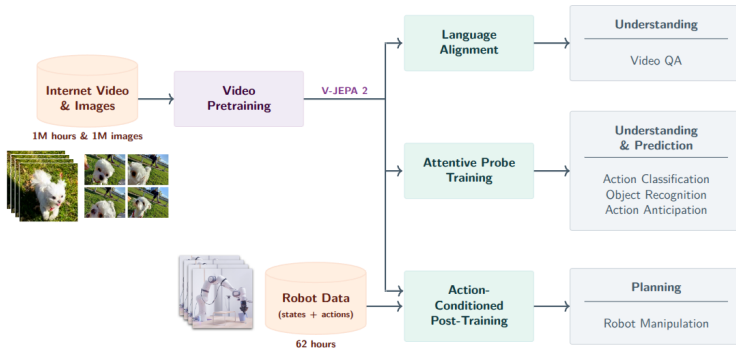
Subhankar Roy

Department of Management, Information and Production Engineering (DIGIP)
University of Bergamo

Recap: Unsupervised Representation Learning

- Under the umbrella of unsupervised learning, representation learning (or **self-supervised learning**) involves **automatically discovering** useful features or **representations** from raw data (eg, images alone) to improve performance on tasks like classification.
- We develop a **pretext task**, or a "fake" objective, that allows us to learn useful features, without requiring class labels. They are:
 - ▶ Rotation prediction
 - ▶ Predict relative patch locations
 - ▶ Solving jigsaw puzzles
 - ▶ Predict missing pixels (or inpainting)
 - ▶ Image coloring, video coloring
 - ▶ A more general pretext task that is based on the idea of **contrasting** *positive pairs* from *negative pairs*

Why Visual Representation Learning is Important?



- We have lots of unlabelled data (eg, images and videos) but very little high-quality data for downstream tasks (eg, robot data with action and state information).
- We want to learn a general-purpose model through visual representation learning (or self-supervised learning) and then transfer this model to the downstream task (eg, robot manipulation) through minimal fine-tuning.

Today

- Cover more modern and widely used self-supervised learning methods.
- Dino model family
- Masked autoencoders
- Joint-embedding predictive architecture

Table of Contents

1. DINO model family

- 1.1 DINO
- 1.2 DINOv2
- 1.3 DINOv3

2. Masked Autoencoder (MAE)

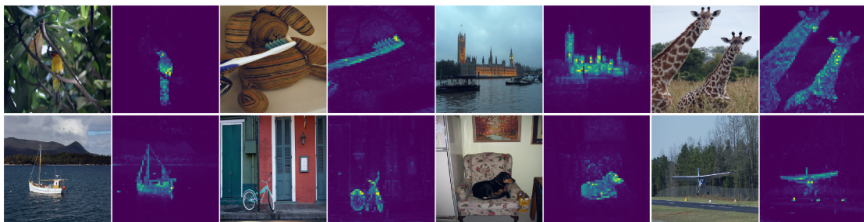
- 2.1 MAE
- 2.2 SatMAE
- 2.3 VideoMAE

3. Joint-Embedding Predictive Architecture (JEPA)

- 3.1 JEPA
- 3.2 I-JEPA
- 3.3 V-JEPA 2

Self-Distillation With No Labels (DINO)

Self-Distillation With No Labels (DINO)

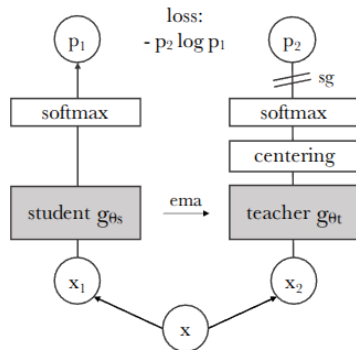


- DINO¹, which stands for self-**dist**illation with **no** labels, is a self-supervised learning algorithm with Vision Transformer (ViT).
- In this work, the authors showed that unlike supervised learning algorithms, ViT when trained with DINO approach yields explicit information about the [semantic segmentation](#) of an image, even when no pixel or class-level labels are available.
- The attention maps corresponding to the [CLS] token are visualized.

¹Dino paper

DINO: Training Algorithm

- Given an input image x , we create two views: x_1 and x_2 using photometric image transformations (eg, random cropping, rotation, jitter).
- We have two networks:
 - ▶ student g_{θ_s}
 - ▶ teacher g_{θ_t}
- Each network makes a prediction (a probability distribution over K outputs) on the corresponding inputs.
- A cross-entropy loss is computed, where the teacher's prediction (p_2) is used as target for the student, which predicts p_1 .

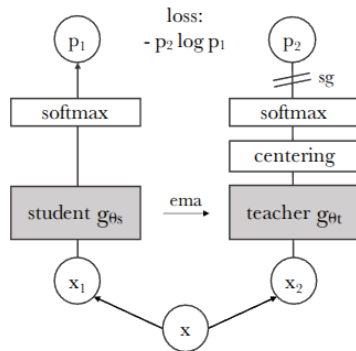


DINO: Training Algorithm

- The loss is backpropagated only through the student network, the teacher receives no gradient (sg= stop gradient).
- Q: How do we update the weights of the teacher?
- A: We update the weights of the teacher using **Exponential Moving Average** (EMA). The weights of the teacher at iteration $i + 1$ is:

$$\theta_t^{(i+1)} = \alpha \theta_t^{(i)} + (1 - \alpha) \theta_s^{(i)}, \alpha \in [0, 1]$$

- The teacher is a smoothed version of the student. Typically, α is close to 1.

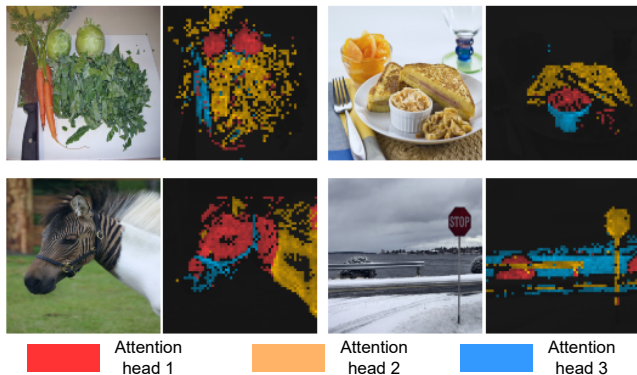


DINO: Preventing Collapse

- In SimCLR during loss computation, we have both positive and the negatives. Recall, that the denominator of the loss function is made up to negatives.
- However, in DINO we do not have any negatives! Because of this, the two networks can predict the same constant distribution and the loss is still minimized, even though learning is useless.
- To prevent this centering and sharpening is used at the output the **teacher network**.
 - ▶ **Centering** prevents one dimension to dominate but encourages the distribution to be uniform (subtracting c , the average of a mini-batch, makes the output flatter).
 - ▶ **Sharpening** makes the output to be peaked (dividing by $\tau \rightarrow 0$ makes the softmaxed output peaked).

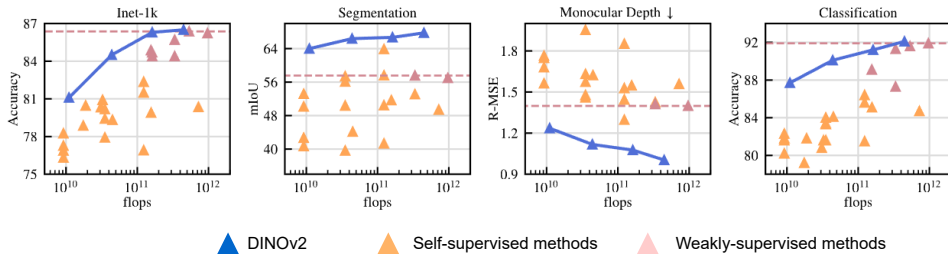
$$p_2 = \text{softmax}\left(\frac{z_t - c}{\tau}\right), c \leftarrow mc + (1 - m) \frac{1}{B} \sum_{i=1}^B g_{\theta_t}(\mathbf{x}_i)$$

DINO: Attention Maps



- We visualize the attention maps (of the last layer) corresponding to the [CLS] token. Different attention heads (denoted by different colors) focus on different locations that represents different objects or parts.

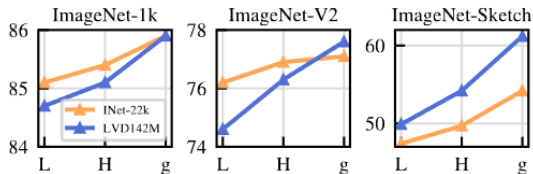
DINOv2



- In the follow-up work DINOv2², the authors explored different ways of improving the performance of the model.
- The findings are that model training is stabilized and performance improves with:
 - ▶ Higher quality unlabelled data
 - ▶ Bigger model size
 - ▶ Improved training losses

²DINOv2 paper

DINOv2: Scaling Laws



- They experimented with ViT of different sizes (or parameter count). It was observed that as the models became bigger, the performance also improved. Same observation for increasing data size: ImageNet-22k (14M images) and LVD-142M (142M images).

Model	Params (approx)	Hidden dim	Layers	Heads	Patch
ViT-Small (ViT-S)	5M	192	12	3	16
ViT-Large (ViT-L)	307M	1024	24	16	16
ViT-Huge (ViT-H)	632M	1280	32	16	14
ViT-Giant (ViT-G)	1.8B	1664	48	16	14

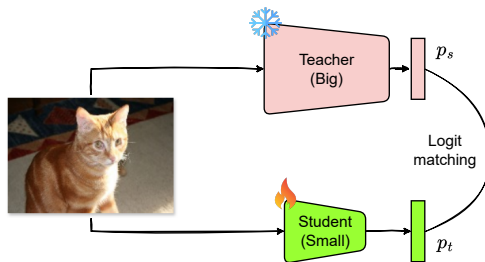
Detour: Knowledge Distillation

- It is clear that bigger the model, higher is the performance.
- However, the bigger the model is, the more memory it will require during inference.
- In resource constrained deployment conditions, we prefer to have smaller models but that perform as well as big models.
- **Knowledge distillation** (KD)³ allows us to distill the knowledge of a big model into a smaller but (almost) equally well-performing model. KD is a very popular technique.



³KD paper

Detour: Knowledge Distillation

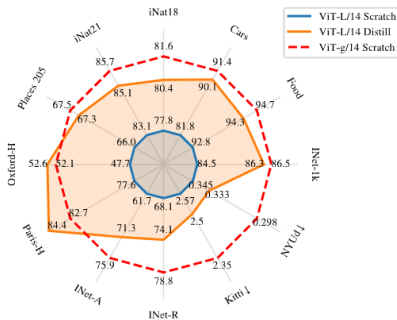


- In KD the teacher is typically a bigger and well-trained neural network, trained on a large dataset. The student is a smaller network, initialized from scratch.
- The teacher logits act as a target, that the student must predict, ie, logit matching.
- The KD loss is the KL divergence loss between the teacher and student probabilities:

$$\text{KL}(p_t \parallel p_s) = \sum_{i \in \mathcal{C}} p_{t,i} \log \frac{p_{t,i}}{p_{s,i}} = \sum_{i \in \mathcal{C}} \left[\underbrace{-p_{t,i} \log p_{s,i}}_{\text{the part that matters}} + p_{t,i} \log p_{t,i} \right]$$

DINOv2: Knowledge Distillation

- It was shown in DINOv2 paper that we can first train a much bigger network (ViT-g/14, a ViT-g with 14×14 patch size) on a large dataset and then distill the knowledge into a smaller network (ViT-L/14).
- The performance of this ViT-L/14-Distill model is better than the ViT-L/14-Scratch model that is trained from scratch, even though they have the same size.
- The performance is reported on several datasets: Oxford-H, Places 205, etc.

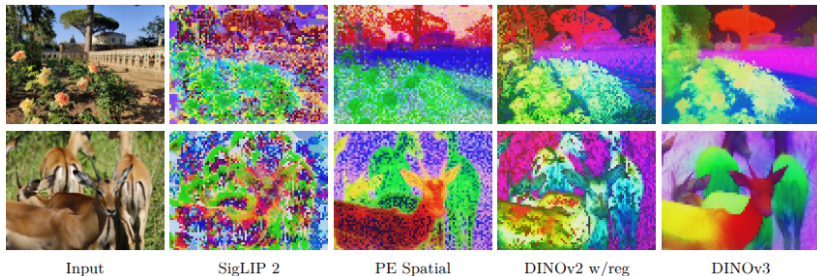


DINOv3

- DINOv3⁴ is a follow-up work of DINOv2, where the authors:
 - ▶ Increased the size of the training set.
 - ▶ Increased the size of the model to 7B. Typically, the image encoders are smaller, less than 1B parameters.
 - ▶ Proposed several architectural changes and improved objective functions.
- The results are quite impressive. This model excels in various computer vision tasks such as image classification, segmentation and others.
- If you want to perform any computer vision task, then use the frozen DINOv3 backbone as the feature extractor. Very likely it will work out-of-the-box, without needing to fine-tune any parameters.

⁴DINOv3 paper

DINOv3

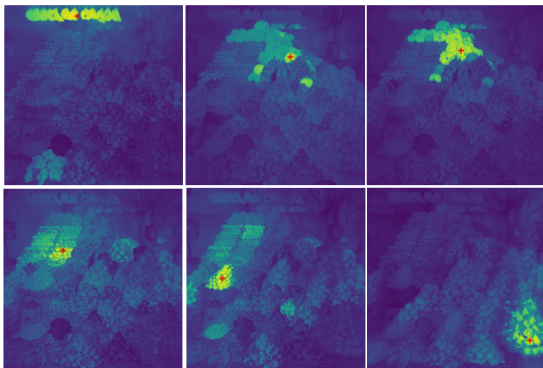


- Images are converted to features using several models. The feature maps are of resolution 80×60 .
- On these features, we run PCA to a 3D space, and then map them to RGB channels. Same color represents same object part.
- DINOv3 although not trained with no class and pixel labels, demonstrates remarkable ability in segmenting different parts of the objects.

DINOv3



Attention Maps



- Attention maps are computed between the patch denoted by the red cross + and all other patches (or tokens) of the input image.
- Similar tokens should be highly activated (more yellow).

Masked Autoencoder (MAE)

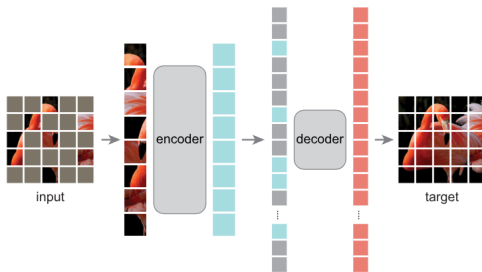
Masked Autoencoder (MAE)



- **Masked autoencoder** (MAE)⁵ is a self-supervised learning algorithm that masks random patches of the input image and reconstructs the missing pixels.
- MAE is similar to the pre-training objective in BERT, where we randomly mask tokens in the input sentence, and ask the network to predict the masked tokens.
- The intuition in MAE is that, to be able to reconstruct an image, the model should learn to encode relevant semantic information from partial information.

⁵MAE paper

MAE: Architecture



- During pre-training, a large random subset of image patches (eg, 75%) is masked out. MAE has two networks:
 - ▶ **Encoder.** The encoder (eg, a ViT) takes as input only the *visible patches* and encodes them into embeddings.
 - ▶ **Decoder.** The decoder takes as input the embeddings of the visible patches, as well as masked tokens. The decoder's job is to reconstruct the original image in pixels.
- After pre-training, the decoder is discarded and the encoder is applied to uncorrupted images (full set of patches) for recognition tasks.

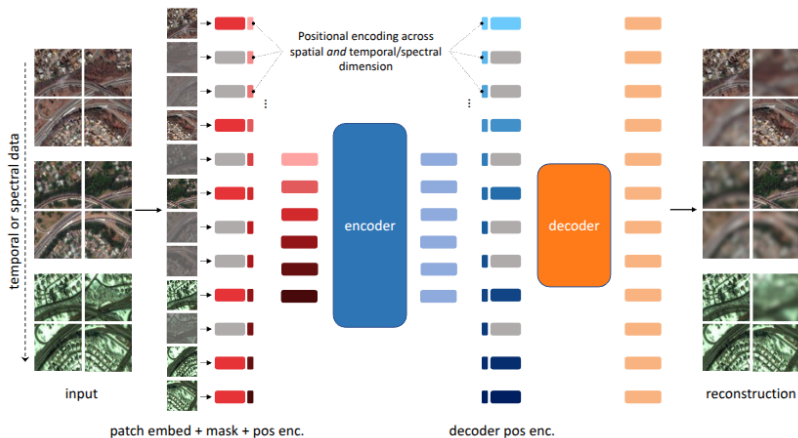
MAE: Applications

- The pre-trained model is then deployed for performing several computer vision downstream tasks. The results in the paper shows the effectiveness of the self-supervised learning strategy in MAE.
- MAE pre-training has been shown to scale well with more data and bigger model size, and therefore explored in many application areas with a lot of unlabelled data, such as:
 - ▶ Remote sensing⁶, where the satellites (eg, Sentinel) acquire hundreds of gigabytes of images of the earth everyday.
 - ▶ Video understanding⁷ since we generate or upload lots of videos on the internet everyday.

⁶SatMAE paper

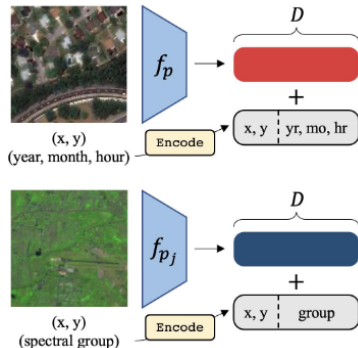
⁷VideoMAE paper

Masked Autoencoder for Satellite Imagery (SatMAE)

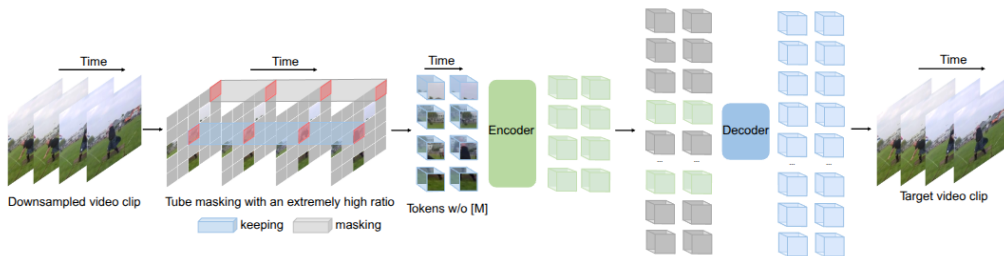


Masked Autoencoder for Satellite Imagery (SatMAE)

- The satellites acquire different kinds of data:
 - ▶ The images of the same geographical location during its revisit at different times during the year → temporal information.
 - ▶ Images of different spectral bands: RGB, Infrared, etc → multi-spectral information.
- We can encode these additional information, ie, time and spectral channels, into the encoder for learning time and band-aware representation.



Masked Autoencoder for Videos: VideoMAE

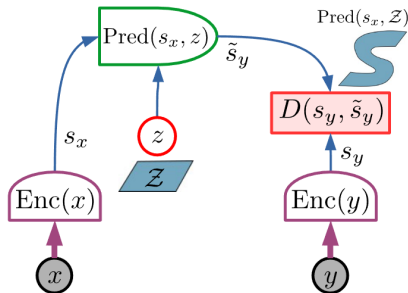


- We can extend the idea of MAE to unlabelled videos too, and the video counterpart is called VideoMAE.
- The task is now masking (90% to 95%) random cubes (the set of tokens at a given location but across frames) and reconstructing the missing ones.
- Through such a pre-training, the model learns spatiotemporal features.

Joint-Embedding Predictive Architecture (JEPA)

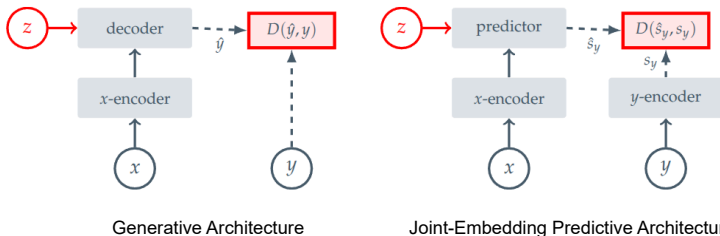
Joint-Embedding Predictive Architecture (JEPA)

- JEPA^a is a general framework for self-supervised representation learning.
- It's goal is to capture the dependencies between x and y , which are compatible inputs, in the **representation space**, instead of generating y from x . For example, x can be the video frames, and y can be the audio of the media.
- It can additionally include a latent variable z to capture the information necessary to predict s_y that is not present in s_x .



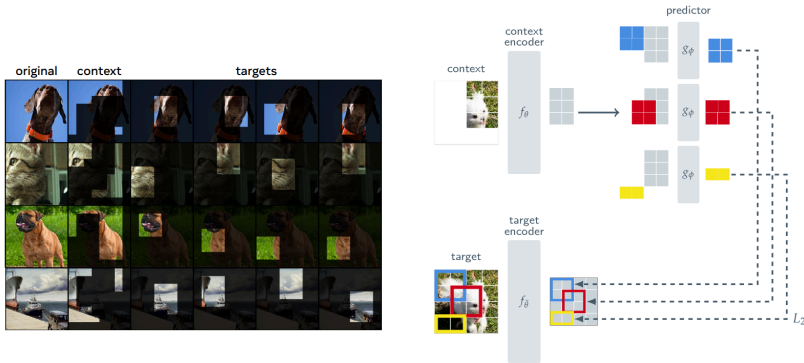
^aJEPA paper (see Sec. 4.5 and 4.5)

Comparing JEPA with Generative Models



- A key difference between generative and JEPA architecture is that in JEPA the loss function is applied in representation/embedding space, not input space.
- JEPAs learn to predict the embeddings of a signal y from a compatible signal x , using a predictor network that is conditioned on an additional (possibly latent) variable z to facilitate prediction.
- The encoders are transformer-based backbones.

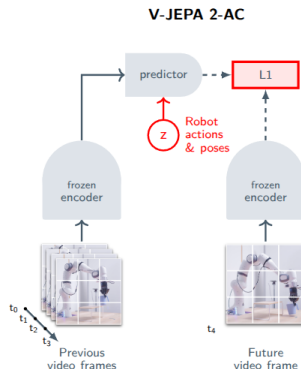
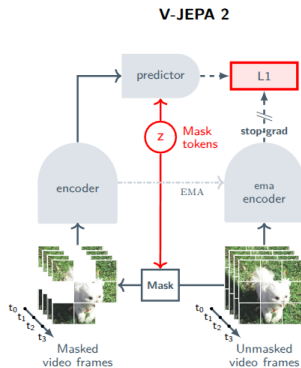
Image-based Joint-Embedding Predictive Architecture (I-JEPA)



- I-JEPA⁸ uses a single context block to predict the representations of various target blocks originating from the same image at a specific location.

⁸I-JEPA paper

Video JEPA: V-JEPA 2



- V-JEPA 2⁹ is first pre-trained on videos, and then later transferred to learn an action controlled model (V-JEPA 2-AC). Such models can be used to anticipate outcomes and plan actions in new environments..

⁹V-JEPA 2 paper

Video JEPA: V-JEPA 2

- A video clip is patchified into a sequence of tokens and a mask is applied by dropping a subset of the tokens. The encoder then processes the masked video sequence and outputs an embedding vector for each input token.
- Next, the outputs of the encoder are concatenated with a set of learnable mask tokens that specify the position of the masked patches, and subsequently processed by the predictor.
- The outputs of the predictor are then regressed to the prediction targets using an L1 loss.
- The prediction targets are computed by an ema-encoder, the weights of which are defined as an exponential moving average of the encoder weights.
- After pretraining, we freeze the video encoder and learn a new action-conditioned predictor, V-JEPA 2-AC, on top of the learned representation. We leverage an autoregressive feature prediction objective that involves predicting the representations of future video frames conditioned on past video frames, actions, and end-effector states.

Conclusions

- Core idea in unsupervised representation learning is to learn useful representations without labels by creating learning signals from the data itself.
- Contrastive Family (SimCLR): Learn by pulling positives together and pushing negatives apart.
- Emerging Paradigm (DINO/DINOv2): Use self-distillation without labels.
- Predictive/Reconstruction Approaches (MAE): Learn by reconstructing masked inputs.
- Generative Latent Modeling (JEPA): Predict representations, not pixels.
- Pre-training is typically expensive, involving thousands of hours of GPU time.